

Evaluación del rendimiento de motores de aeronaves en despegue mediante Machine Learning

Presentación y análisis del artículo de Bulent Kurt (2024, *Neural Computing and Applications*)

UNAQ IECSA, 10997@soyunaq.mx

20 de julio de 2026



- 1 Contexto y motivación
- 2 Fundamentos de Machine Learning
- 3 Metodología del estudio
- 4 Resultados del artículo
- 5 Conclusiones y cierre

- 1 Contexto y motivación
- 2 Fundamentos de Machine Learning
- 3 Metodología del estudio
- 4 Resultados del artículo
- 5 Conclusiones y cierre

¿Por qué importa el desempeño de un motor en el despegue?

Durante el despegue, los motores de un avión trabajan a su **velocidad y temperatura máximas** para generar el empuje necesario. Este es el punto de operación más exigente de todo el vuelo.

Con el uso y el tiempo, el motor se **degrada**, es decir, deja de comportarse como cuando era nuevo, debido a:

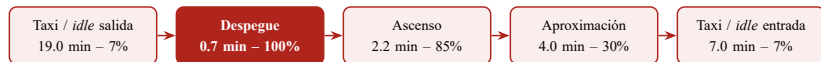
- *Fouling* (ensuciamiento de álabes),
- Erosión y abrasión,
- Corrosión y oxidación en caliente,
- Degradación mecánica general.

Consecuencia clave

Un motor degradado **consume más combustible y genera más emisiones contaminantes** (CO_2 , NO_x , hollín, etc.) para producir el mismo empuje.

El ciclo LTO (*Landing and Take-Off*) de la ICAO

La ICAO (Organización de Aviación Civil Internacional) define un ciclo estándar, el **ciclo LTO**, para medir y certificar emisiones cerca de los aeropuertos (por debajo de 3000 pies de altitud). Tiene 4 fases, cada una con una duración y un porcentaje de empuje fijos:



En **rojo**, la fase que estudia este artículo: el despegue, con el motor al 100% de empuje. (Datos: Tabla 1 del artículo, según ICAO Annex 16).

El banco de datos EEDB y el vacío en la literatura

Todo motor turborreactor o turbofán certificado por la ICAO debe pasar pruebas en banco, cuyos resultados se guardan públicamente en el **EEDB** (*ICAO Engine Emissions Data Bank*): miles de registros con parámetros de operación y emisiones de motores reales.

El problema que aborda el artículo

Hoy en día, evaluar si un motor se está degradando requiere **desarmarlo** o consultar a un **experto**. **No existe** en la literatura un modelo que evalúe el desempeño del motor en el despegue usando únicamente los datos del EEDB y técnicas de *Machine Learning* (ML).

Objetivo del artículo: construir un modelo de ML que prediga el parámetro ***fuel flow T/O*** (flujo de combustible en despegue, en kg/s) a partir de variables fáciles de medir, y usarlo como indicador indirecto del estado del motor.

¿Por qué el *fuel flow T/O* como indicador de desempeño?

El ***fuel flow T/O*** es la cantidad de combustible que quema el motor por segundo durante el despegue. Es una variable que:

- Se usa directamente para calcular las **emisiones** del motor (junto con los índices de emisión),
- **Cambia** cuando hay fallas o degradación: separación del motor, apagado en vuelo, daño severo, problemas de control de combustible, fugas, etc.
- Está **disponible** para todos los motores certificados en el EEDB.

Idea del método

Si el modelo predice cuánto **debería** fluir el combustible dadas las condiciones de un motor *sano*, y el valor real medido se aleja demasiado de esa predicción, eso es evidencia de una posible degradación del motor.

1 Contexto y motivación

2 Fundamentos de Machine Learning

Regresión de Procesos Gaussianos (GPR)

Máquinas de Vectores de Soporte (SVM)

Perceptrón Multicapa (MLP)

3 Metodología del estudio

4 Resultados del artículo

5 Conclusiones y cierre

¿Qué es un problema de regresión supervisada?

Antes de entrar en los tres métodos del artículo, aclaremos la tarea que resuelven los tres: es un problema de **regresión supervisada**.

- Tenemos ejemplos pasados donde **conocemos** tanto las variables de entrada (edad del diseño del motor, condiciones ambientales, etc.) como el valor real de la variable a predecir (*fuel flow T/O*).
- El algoritmo **aprende** una función f tal que $\hat{y} = f(x_1, x_2, \dots, x_n)$ se parezca lo más posible al valor real y .
- Como la salida es un **número continuo** (kg/s) y no una categoría, es **regresión** y no clasificación.

Los tres modelos del artículo —**GPR**, **SVM** y **MLP**— son tres formas distintas de construir esa función f a partir de los datos.

GPR: la idea intuitiva

La **Regresión de Procesos Gaussianos** (*Gaussian Process Regression*, GPR) no busca “una sola curva óptima”, sino que trabaja con una **distribución de probabilidad sobre funciones posibles**.

- Antes de ver datos, el modelo considera **muchas** funciones plausibles.
- Cada punto de dato observado **descarta** las funciones que no pasan cerca de él.
- El resultado es una función media (la predicción) **más una banda de incertidumbre**: el modelo también te dice *qué tan seguro está* de su predicción en cada zona.

Esto es justo lo que se usa después en el artículo para construir los **intervalos de confianza** que detectan degradación del motor (Sección de resultados).

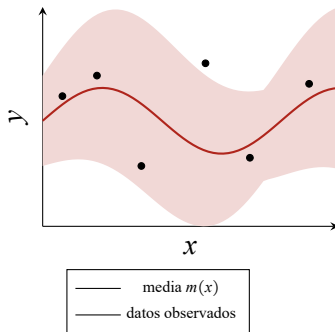
GPR: función media, incertidumbre y kernel

Formalmente (Ec. 3 del artículo):

$$f(x) \sim \text{GPR}(m(x), k(x, x'))$$

- $m(x)$: función **media**, la mejor predicción puntual.
- $k(x, x')$: función de **covarianza** (*kernel*): mide cuán “parecidos” son dos puntos x y x' ; define qué tan suave es la incertidumbre.

El artículo probó 3 *kernels*:
exponencial, exponencial cuadrado y racional cuadrático.



Esquema ilustrativo: la banda se angosta cerca de los datos observados y se ensancha donde hay poca información.

SVM: la idea intuitiva

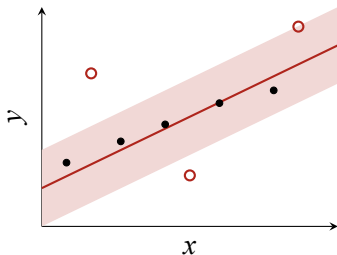
Las **Máquinas de Vectores de Soporte** (*Support Vector Machines*, SVM) nacieron para **clasificar** (separar dos grupos con la mejor frontera posible), pero también se pueden usar para **regresión**: en ese caso, en vez de separar clases, buscan la función que mejor se ajusta a los datos.

- Se define un **tubo** de ancho ε alrededor de la función ajustada.
- Los puntos que caen **dentro** del tubo no penalizan al modelo (se toleran como ruido normal).
- Los puntos que quedan **fuera** del tubo son los que realmente definen la forma de la función: se llaman **vectores de soporte**.

Nota sobre el nombre

En tareas de regresión, esta técnica a veces se llama *Support Vector Regression* (SVR). El artículo de Kurt la llama **SVM** en todo momento, y esta presentación mantiene ese mismo nombre por consistencia con la fuente.

SVM: el tubo ε -insensible y los vectores de soporte

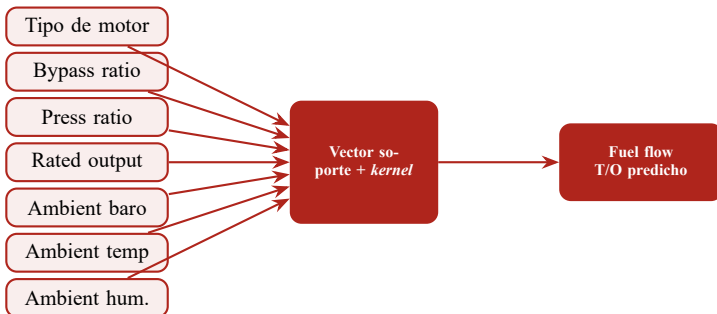


- **Puntos negros:** dentro del tubo $\pm\varepsilon$, no penalizan.
- **Círculos rojos:** fuera del tubo, son los **vectores de soporte** que “sostienen” la forma de la función.
- Si los datos no son lineales, se usa una **función kernel** para proyectarlos a un espacio donde sí lo sean (*kernel trick*).

Esquema ilustrativo. El artículo probó *kernels* **lineal**, **cuadrático** y **cúbico**.

SVM: estructura del modelo usado en el artículo

Ecuación general del método con *kernel* (Ec. 4): $y = (K_{xi} * W_{jk}) + b$, donde K_{xi} es la función *kernel* no lineal aplicada a la entrada, W_{jk} el vector de pesos y b el sesgo (*bias*).

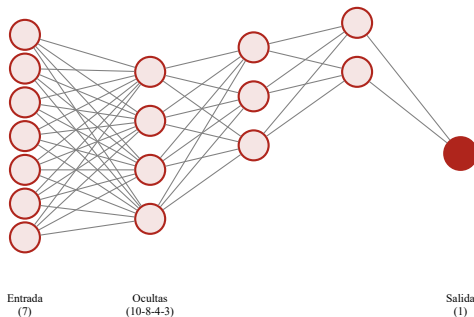


MLP: la neurona artificial y la red

El **Perceptrón Multicapa** (*Multilayer Perceptron*, MLP) es un tipo de **red neuronal artificial**, inspirada (de forma muy simplificada) en cómo se conectan las neuronas biológicas.

- Cada **neurona artificial** recibe varias entradas, las multiplica por **pesos**, las suma, y aplica una **función de activación** no lineal (p. ej. *tansig*, *logsig*, *purelin*).
- Las neuronas se organizan en **capas**: capa de entrada, una o más capas **ocultas**, y capa de salida.
- El modelo **aprende** ajustando los pesos mediante **retropropagación** (*backpropagation*): compara la predicción con el valor real y corrige los pesos hacia atrás, capa por capa.

MLP: arquitectura usada en el artículo



Esquema simplificado de la red real (7 entradas → capas ocultas de tamaño 10, 8, 4 y 3 → 1 salida). Activaciones: *tansig–logsig–tansig–purelin*. Entrenada con **trainlm** (Levenberg–Marquardt), y comparada con **trainrp** (retropropagación resiliente) y **traincgb** (gradiente conjugado con reinicios de Powell–Beale).

Resumen: intuición de los tres métodos

Método	Idea central	Punto fuerte típico
GPR	Distribución sobre funciones; predicción + incertidumbre	Da bandas de confianza de forma natural
SVM	Tubo ϵ -insensible + vectores de soporte + <i>kernel</i>	Robusto con pocos datos y ruido moderado
MLP	Capas de neuronas con pesos ajustados por <i>backpropagation</i>	Captura relaciones muy no lineales y complejas

Los tres se entrenaron y compararon **con exactamente los mismos datos** para que la comparación sea justa.

- 1 Contexto y motivación
- 2 Fundamentos de Machine Learning
- 3 Metodología del estudio**
- 4 Resultados del artículo
- 5 Conclusiones y cierre

Fuente de datos: el EEDB en dos cortes temporales

Se usaron dos extractos del EEDB:

- **EEDB-09/2019:** 565 registros → se usa para **entrenar, validar y probar** los tres modelos.
- **EEDB-07/2021:** 773 registros → se usa **solo al final**, para comprobar que el mejor modelo generaliza a datos nuevos.

Tabla 1: Estadísticos descriptivos de las variables (Tablas 4 y 5 del artículo)

Variable	EEDB-09/2019 ($n = 565$)				EEDB-07/2021 ($n = 773$)			
	Mín	Máx	Media	DE	Mín	Máx	Media	DE
Tipo de motor	1.00	2.00	1.2248	0.4178	1.00	2.00	1.2096	0.4073
Bypass ratio	0.64	12.72	6.1787	2.3911	0.00	12.72	6.5978	2.6412
Press ratio	9.76	49.40	30.4341	8.3014	9.76	49.60	31.3034	8.5283
Rated output (kN)	9.79	513.90	189.7491	116.318	9.79	513.90	186.5325	118.293
Ambient temp (K)	144.0	308.50	286.9116	10.306	226.5	308.50	287.1879	8.146
Ambient humidity	0.00	0.02	0.0069	0.0037	0.00	0.04	0.0070	0.0038
Fuel flow T/O	0.15	4.69	1.7095	0.9479	0.15	4.69	1.6410	0.9635

Selección de variables de entrada

Antes de entrenar los modelos de ML, el autor hizo un **análisis de regresión múltiple** para confirmar cuáles variables realmente influyen en el *fuel flow* T/O.

Resultado del análisis (Tablas 7 y 8)

- $R = 0.990$, $R^2 = 0.979$ (el modelo explica el 97.9% de la variación),
- Prueba ANOVA: Sig. = $0.000 < 0.001 \Rightarrow$ el modelo es **altamente significativo**.

7 variables seleccionadas

Tipo de motor, *bypass ratio*, *press ratio*, *rated output*, presión ambiente, temperatura ambiente y humedad ambiente.

Estas 7 variables son las que después alimentan a GPR, SVM y MLP como entradas (ver diagramas de las Figuras 4 y 5 del artículo).

La ecuación de regresión múltiple resultante

$$y = \beta_0 + \beta_1 p_1 + \beta_2 p_2 + \cdots + \beta_n p_n + \epsilon \quad (\text{Ec. 1})$$

Con los coeficientes estimados (Tabla 9), la ecuación queda (Ec. 2):

$$\begin{aligned} \text{Fuel Flow } T/O = & 1.302 - 0.49 \times \text{tipo de motor} - 0.1 \times \text{bypass ratio} \\ & - 7.308 \times 10^{-5} \times \text{press ratio} + 0.009 \times \text{rated output} \\ & + 2.242 \times 10^{-7} \times \text{ambient baro} - 0.002 \times \text{ambient temp} \\ & - 4.612 \times \text{ambient humidity} \end{aligned}$$

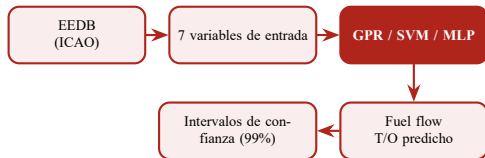
Lectura rápida de signos: a mayor *rated output* (motor más potente), mayor flujo

de combustible ($\beta > 0$); a mayor *bypass ratio* (motores más eficientes/modernos), menor flujo de combustible ($\beta < 0$). Esta regresión **no es** uno de los tres modelos de ML: solo sirvió para **validar** qué variables usar como entrada.

Partición de datos y flujo general del estudio

- **70%** entrenamiento (395 registros),
- **15%** validación (85 registros),
- **15%** prueba (85 registros).

La partición es **la misma** para los tres modelos, para que la comparación final sea justa.



Métricas para comparar los modelos

Los tres modelos se comparan con las mismas tres métricas de error (más bajo = mejor), calculadas sobre n observaciones, donde $FF_{T/O}$ es el valor real y $\hat{f}_{T/O}$ el valor predicho:

$$MSE = \frac{1}{n} \sum_{i=1}^n (FF_{T/O} - \hat{f}_{T/O})^2 \quad (\text{Ec. 5, error cuadrático medio})$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |FF_{T/O} - \hat{f}_{T/O}| \quad (\text{Ec. 6, error absoluto medio})$$

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{FF_{T/O} - \hat{f}_{T/O}}{FF_{T/O}} \right| \quad (\text{Ec. 7, error porcentual absoluto medio})$$

Criterio de calidad (Lewis, 1982)

Un modelo con $MAPE < 10\%$ se considera de **alta precisión**; los autores citan que valores por debajo de ese umbral se consideran un modelo “muy bueno”.

- 1 Contexto y motivación
- 2 Fundamentos de Machine Learning
- 3 Metodología del estudio
- 4 Resultados del artículo**
Resultados por modelo
- 5 Conclusiones y cierre

Resultados: GPR

Se probaron 3 *kernels*; se comparan por MSE en el conjunto de prueba (Tabla 10):

Kernel	R (prueba)	MSE (prueba)
Exponencial	0.9964	0.0066
Racional cuadrático	0.9953	0.0090
Exponencial cuadrado	0.9856	0.0272

Estructura del modelo ganador (Tabla 11): función base constante, escala de kernel automática, $\sigma = 0.0096$, optimizador *Quasi-Newton*. En entrenamiento llega a $R = 1$ y $MSE \approx 1.3 \times 10^{-6}$: ajusta casi perfecto los datos ya vistos, pero eso no garantiza que generalice igual de bien (por eso importan validación y prueba).

Resultados: SVM

Se probaron 3 *kernels*; se comparan por MSE en el conjunto de prueba (Tabla 12):

Kernel	R (prueba)	MSE (prueba)	
Cuadrático	0.99709	0.00542	(ganador)
Lineal	0.98906	0.02062	
Cúbico	0.99459	0.01016	

Estructura del modelo ganador (Tabla 13): función polinomial, escala de kernel automática, restricción de caja (*box constraint*) = 1.2076, sesgo = 1.7458, $\varepsilon = 0.1208$ (ancho del tubo insensible). Nota: el SVM **cúbico** tuvo un desempeño muy pobre en validación ($R = 0.489$), lo que muestra que un kernel más complejo no siempre generaliza mejor.

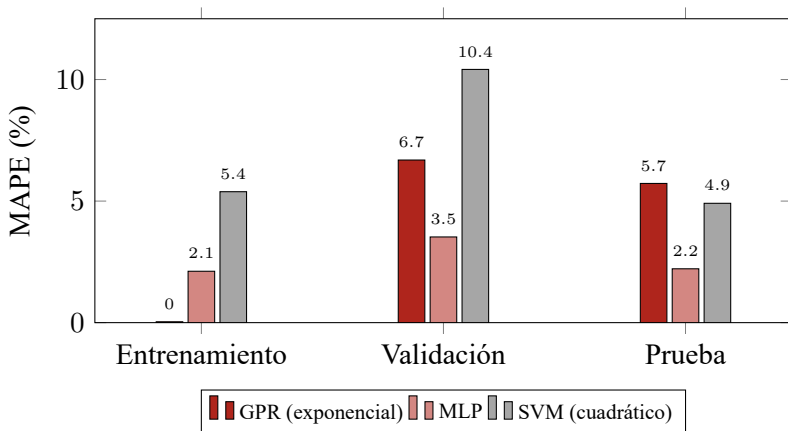
Resultados: MLP

Se entrenaron 9 configuraciones con 3 algoritmos distintos; se muestran las más relevantes (Tabla 14, ordenadas por MSE de prueba):

Modelo	Algoritmo	R (prueba)	MSE (prueba)	
Modelo 1	trainlm	0.9986	0.0025488	(ganador)
Modelo 3	trainlm	0.9972	0.0057155	
Modelo 5	trainrp	0.9952	0.0079494	
Modelo 8	traincgb	0.99469	0.01059	

Arquitectura ganadora (Tabla 15): 7 entradas $\rightarrow$ capas ocultas 10-8-4-3 $\rightarrow$ 1 salida, activaciones *tansig-logsig-tansig-purelin*, entrenada con **trainlm** (Levenberg–Marquardt), sobre la misma partición 70/15/15%.

Comparación final: ¿cuál modelo gana?



Datos reales de la Tabla 16. GPR se ajusta casi perfecto en entrenamiento pero pierde precisión en validación/prueba (señal de sobreajuste leve); **MLP** es el más equilibrado: mejor en validación y prueba, y todos sus MAPE quedan bajo 5%.

Prueba de generalización con datos de 2021

El mejor modelo global, el **MLP**, se volvió a evaluar —sin reentrenarlo— con 773 registros nuevos del EEDB-07/2021, que no participaron en el entrenamiento:

Conjunto de datos	n	MAE	MSE	R	MAPE
EEDB-09/2019 (original)	565×8	0.0294	0.0022	0.99878	2.341%
EEDB-07/2021 (nuevo)	773×8	0.0298	0.0219	0.98839	2.897%

Interpretación

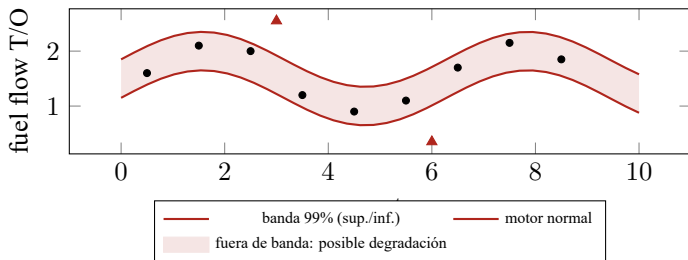
Con datos completamente nuevos, el MAPE apenas sube de 2.34% a 2.90%, muy por debajo del umbral de 10%. Esto respalda que el modelo **generaliza bien** y no solo memorizó los datos de entrenamiento.

Intervalos de confianza para detectar degradación

Con el MLP ya entrenado, se construye un intervalo de confianza al **99%** alrededor de cada predicción (Ec. 8):

$$\hat{f}_{T/O} - z_{\frac{\alpha}{2}} \frac{\sigma}{\sqrt{n}} < FF_{T/O} < \hat{f}_{T/O} + z_{\frac{\alpha}{2}} \frac{\sigma}{\sqrt{n}}$$

Con $z_{\frac{\alpha}{2}} = 2.58$ para el 99% de confianza (Tabla 18).



Esquema ilustrativo (no son los datos reales de las Figuras 10-11). Si el valor medido cae **dentro** de la banda, el motor se considera normal; si cae **fuera**, se marca como posible anomalía de desempeño.

- 1 Contexto y motivación
- 2 Fundamentos de Machine Learning
- 3 Metodología del estudio
- 4 Resultados del artículo
- 5 Conclusiones y cierre**

Hallazgos principales

- El **MLP** (modelo 1, entrenado con *trainlm*) fue el mejor modelo global: mejor desempeño en validación y prueba, con MAPE $< 5\%$ en todos los conjuntos.
- El **GPR exponencial** logró el mejor ajuste en entrenamiento, pero con más error en validación/prueba que el MLP.
- El **SVM cuadrático** fue el mejor de los tres *kernels* de SVM, pero tuvo los errores más altos de los tres modelos finalistas.
- El MLP **generalizó bien** a datos de un año distinto (2021), con MAPE de solo 2.90%.
- Se construyó un sistema de **intervalos de confianza (99%)** que permite señalar posibles degradaciones sin desarmar el motor.

Aporte práctico del estudio

¿Qué permite este modelo que antes no era posible?

Evaluar el desempeño de un motor en el despegue **sin desmontarlo** y **sin requerir un experto**, usando solo datos ya disponibles en el EEDB.

Esto puede ayudar a reducir:

- El consumo de combustible innecesario,
- Los costos de mantenimiento (al detectar degradación temprano),
- El costo total del vuelo,
- El riesgo de accidentes asociado a fallas no detectadas,
- Las emisiones contaminantes de la aviación.

Discusión crítica (reflexión propia)

Fortalezas

Comparación rigurosa y transparente de 3 familias de modelos con la misma partición de datos; validación real con un segundo dataset (2021); uso de intervalos de confianza con una interpretación práctica clara.

Puntos a considerar

- Los datos provienen de una **única fuente institucional** (EEDB/ICAO); no se prueba con datos de otras bases o de vuelo real (FDR).
- La validación de generalización usa un solo año adicional (2021), de la misma base de datos.
- No se comparan con modelos de *deep learning* más recientes (p. ej. redes profundas, *ensembles*), que otros trabajos de la Tabla 3 sí exploran.

Estos puntos no invalidan el estudio; son posibles líneas de trabajo futuro.

Referencias clave

- Kurt, B. (2024). Evaluation of aircraft engine performance during takeoff phase with machine learning methods. *Neural Computing and Applications*, 36, 19173–19190.
- Cortes, C., Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.
- Lewis, C. D. (1982). *Industrial and Business Forecasting Methods*. Butterworths.
- ICAO (2019, 2021). ICAO Aircraft Engine Emissions Databank (EEDB).
- Yildirim, M.T., Kurt, B. (2019). Confidence interval prediction of ANN estimated LPT parameters. *Aircraft Engineering and Aerospace Technology*, 92(2), 101–106.

Gracias!